

# Rajalakshmi Engineering College

Name: Nithilan M  
Email: 241001155@rajalakshmi.edu.in  
Roll no: 241001155  
Phone: 8122020972  
Branch: REC  
Department: IT - Section 4  
Batch: 2028  
Degree: B.E - IT

Scan to verify results



## 2024\_28\_III\_OOPS Using Java Lab

### REC\_2028\_OOPS using Java\_Week 5\_CY

Attempt : 1  
Total Mark : 40  
Marks Obtained : 40

#### Section 1 : Coding

##### 1. Problem Statement

Arjun is working as a developer for CityWater Supply Board, which wants to build a household water billing system.

Each household's water account has:

A Customer ID (integer) A Customer Name (string) Liters Consumed (double)

The water bill is calculated based on these rules:

For the first 500 liters 2 per liter For the next 500 liters (501–1000) 3 per liter For liters above 1000 5 per liter If the total bill exceeds 3000, a 10% discount is applied on the final bill.

Arjun has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Liters Consumed (double).

### ***Output Format***

For each customer, print the details in the following format:

Customer ID: <customer\_id>

Customer Name: <customer\_name>

Final Bill: <final\_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

300

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 600.0

### ***Answer***

```
import java.util.*;
```

```
class Customer {
    private int customerId;
    private String customerName;
    private double litersConsumed;
    private double finalBill;

    public Customer(int customerId, String customerName, double
    litersConsumed) {
        this.customerId = customerId;
        this.customerName = customerName;
        this.litersConsumed = litersConsumed;
        this.finalBill = calculateBill(litersConsumed);
    }

    public int getCustomerId() {
        return customerId;
    }

    public String getCustomerName() {
        return customerName;
    }

    public double getLitersConsumed() {
        return litersConsumed;
    }

    public double getFinalBill() {
        return finalBill;
    }

    public void setCustomerId(int customerId) {
        this.customerId = customerId;
    }

    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    public void setLitersConsumed(double litersConsumed) {
        this.litersConsumed = litersConsumed;
        this.finalBill = calculateBill(litersConsumed);
    }
}
```

```

    }

    private double calculateBill(double liters) {
        double amount = 0;

        if (liters <= 500) {
            amount = liters * 2;
        } else if (liters <= 1000) {
            amount = 500 * 2 + (liters - 500) * 3;
        } else {
            amount = 500 * 2 + 500 * 3 + (liters - 1000) * 5;
        }

        if (amount > 3000) {
            amount = amount - (amount * 0.10);
        }

        return amount;
    }

    public void display() {
        System.out.println("Customer ID: " + customerId);
        System.out.println("Customer Name: " + customerName);
        System.out.printf("Final Bill: %.1f%n", finalBill);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());

        for (int i = 0; i < N; i++) {
            int customerId = Integer.parseInt(sc.nextLine().trim());
            String customerName = sc.nextLine().trim();
            double litersConsumed = Double.parseDouble(sc.nextLine().trim());

            Customer customer = new Customer(customerId, customerName,
            litersConsumed);
            customer.display();
        }
    }
}

```

```
        sc.close();  
    }  
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Anjali is working as a developer for the City Basketball Association, which wants to build a system to track and find the top scorer among basketball players.

Each player's record has:

Player ID (integer) Player Name (string) An array of points scored in 5 matches (integers)

The system must calculate:

The total score of each player (sum of all match points). Identify the highest scorer among all players. If two or more players have the same total score, the one with the lower Player ID is considered the top scorer.

Anjali has been asked to implement this system using:

A class with attributes for player details. A constructor to initialize player details. Getter and Setter methods to retrieve and update player details if required. A method to calculate the total score. Objects of the class to represent players.

Finally, display each player's details and announce the Top Scorer.

### **Input Format**

The first line of input contains an integer N (number of players).

For each player:

- The next line contains the Player ID (integer).
- The following line contains the Player Name (string).
- The next line contains 5 integers separated by spaces (points scored in 5

matches).

### **Output Format**

For each player the output prints the following details:

- Player ID: <player\_id>
- Player Name: <player\_name>
- Total Score: <total\_score>

Finally, print "Top Scorer: <player\_name> with <total\_score> points"

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1001

Ravi Kumar

10 20 30 40 50

Output: Player ID: 1001

Player Name: Ravi Kumar

Total Score: 150

Top Scorer: Ravi Kumar with 150 points

### **Answer**

```
import java.util.*;
```

```
class Player {  
    private int playerId;  
    private String playerName;  
    private int[] points;  
    private int totalScore;
```

```
    public Player(int playerId, String playerName, int[] points) {  
        this.playerId = playerId;  
        this.playerName = playerName;  
        this.points = points;
```

```

        calculateTotalScore();
    }

    public int getPlayerId() {
        return playerId;
    }

    public String getPlayerName() {
        return playerName;
    }

    public int getTotalScore() {
        return totalScore;
    }

    public void setPoints(int[] points) {
        this.points = points;
        calculateTotalScore();
    }

    private void calculateTotalScore() {
        int sum = 0;
        for (int p : points) {
            sum += p;
        }
        totalScore = sum;
    }

    public void display() {
        System.out.println("Player ID: " + playerId);
        System.out.println("Player Name: " + playerName);
        System.out.println("Total Score: " + totalScore);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());

        List<Player> players = new ArrayList<>();
        Player topScorer = null;
    }
}

```

```

for (int i = 0; i < N; i++) {
    int playerId = Integer.parseInt(sc.nextLine().trim());
    String playerName = sc.nextLine().trim();
    String[] pointsStr = sc.nextLine().trim().split("\\s+");
    int[] points = new int[5];
    for (int j = 0; j < 5; j++) {
        points[j] = Integer.parseInt(pointsStr[j]);
    }

    Player player = new Player(playerId, playerName, points);
    players.add(player);

    if (topScorer == null ||
        player.getTotalScore() > topScorer.getTotalScore() ||
        (player.getTotalScore() == topScorer.getTotalScore() &&
        player.getPlayerId() < topScorer.getPlayerId())) {
        topScorer = player;
    }
}

for (Player player : players) {
    player.display();
}

System.out.println("Top Scorer: " + topScorer.getPlayerName() + " with " +
topScorer.getTotalScore() + " points");
sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Anjali is now working as a developer for the City Marathon Association, which wants to build a system to track and find the fastest runner among marathon participants.

Each runner's record has:



Runner ID (integer) Runner Name (string) An array of times (in minutes) taken in 5 marathon events (integers)

The system must calculate:

The average time of each runner (sum of all times / 5). Identify the fastest runner (the one with the lowest average time). If two or more runners have the same average time, the one with the lower Runner ID is considered the fastest runner.

Anjali has been asked to implement this system using:

A class with attributes for runner details. A constructor to initialize runner details. Getter and Setter methods to retrieve and update runner details if required. A method to calculate the average time. Objects of the class to represent runners.

Finally, display each runner's details and announce the Fastest Runner.

### ***Input Format***

The first line of input contains an integer N (number of runners).

For each runner:

- The next line contains the Runner ID (integer).
- The following line contains the Runner Name (string).
- The next line contains 5 integers separated by spaces (times in minutes for 5 marathon events).

### ***Output Format***

For each runner the output prints the following details:

- Runner ID: <runner\_id>
- Runner Name: <runner\_name>
- Average Time: <average\_time>

Finally, print "Fastest Runner: <runner\_name> with <average\_time> minutes"

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1001

Ravi Kumar

240 250 245 255 260

Output: Runner ID: 1001

Runner Name: Ravi Kumar

Average Time: 250

Fastest Runner: Ravi Kumar with 250 minutes

### **Answer**

```
import java.util.*;
```

```
class Runner {  
    private int runnerId;  
    private String runnerName;  
    private int[] times;  
    private double averageTime;  
  
    public Runner(int runnerId, String runnerName, int[] times) {  
        this.runnerId = runnerId;  
        this.runnerName = runnerName;  
        this.times = times;  
        calculateAverageTime();  
    }  
  
    public int getRunnerId() {  
        return runnerId;  
    }  
  
    public String getRunnerName() {  
        return runnerName;  
    }  
  
    public double getAverageTime() {  
        return averageTime;  
    }  
}
```

```

    public void setTimes(int[] times) {
        this.times = times;
        calculateAverageTime();
    }

    private void calculateAverageTime() {
        int sum = 0;
        for (int t : times) {
            sum += t;
        }
        averageTime = sum / (double) times.length;
    }

    public void display() {
        System.out.println("Runner ID: " + runnerId);
        System.out.println("Runner Name: " + runnerName);
        System.out.println("Average Time: " + (int)averageTime);
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());

        List<Runner> runners = new ArrayList<>();
        Runner fastest = null;

        for (int i = 0; i < N; i++) {
            int runnerId = Integer.parseInt(sc.nextLine().trim());
            String runnerName = sc.nextLine().trim();
            String[] timeStr = sc.nextLine().trim().split("\\s+");
            int[] times = new int[5];
            for (int j = 0; j < 5; j++) {
                times[j] = Integer.parseInt(timeStr[j]);
            }

            Runner r = new Runner(runnerId, runnerName, times);
            runners.add(r);

            if (fastest == null ||

```

```

        r.getAverageTime() < fastest.getAverageTime() ||
        (r.getAverageTime() == fastest.getAverageTime() && r.getRunnerId() <
        fastest.getRunnerId())) {
            fastest = r;
        }
    }

    for (Runner r : runners) {
        r.display();
    }

    System.out.println("Fastest Runner: " + fastest.getRunnerName() + " with " +
    (int)fastest.getAverageTime() + " minutes");
    sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Meera is working as a developer for CityGas Supply Board, which wants to build a household gas billing system.

Each household's gas account has:

A Customer ID (integer) A Customer Name (string) Units Consumed in cubic meters (double)

The gas bill is calculated based on these rules:

For the first 50 units    4 per unit For the next 100 units (51–150)    6 per unit  
 For units above 150    8 per unit If the total bill exceeds 2000, a 15% discount is applied on the final bill.

Meera has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

### ***Output Format***

For each customer, print the details in the following format:

Customer ID: <customer\_id>

Customer Name: <customer\_name>

Final Bill: <final\_bill> (The final bill must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

30

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 120.0

### ***Answer***

```
import java.util.*;
```

```
class Customer {  
    private int customerId;  
    private String customerName;
```

```
private double unitsConsumed;  
private double finalBill;
```

```
public Customer(int customerId, String customerName, double  
unitsConsumed) {  
    this.customerId = customerId;  
    this.customerName = customerName;  
    this.unitsConsumed = unitsConsumed;  
    this.finalBill = calculateBill(unitsConsumed);  
}
```

```
public int getCustomerId() {  
    return customerId;  
}
```

```
public String getCustomerName() {  
    return customerName;  
}
```

```
public double getUnitsConsumed() {  
    return unitsConsumed;  
}
```

```
public double getFinalBill() {  
    return finalBill;  
}
```

```
public void setCustomerId(int customerId) {  
    this.customerId = customerId;  
}
```

```
public void setCustomerName(String customerName) {  
    this.customerName = customerName;  
}
```

```
public void setUnitsConsumed(double unitsConsumed) {  
    this.unitsConsumed = unitsConsumed;  
    this.finalBill = calculateBill(unitsConsumed);  
}
```

```
private double calculateBill(double units) {  
    double amount = 0;
```

```

        if (units <= 50) {
            amount = units * 4;
        } else if (units <= 150) {
            amount = 50 * 4 + (units - 50) * 6;
        } else {
            amount = 50 * 4 + 100 * 6 + (units - 150) * 8;
        }

        if (amount > 2000) {
            amount = amount - (amount * 0.15);
        }

        return amount;
    }

    public void display() {
        System.out.println("Customer ID: " + customerId);
        System.out.println("Customer Name: " + customerName);
        System.out.printf("Final Bill: %.1f%n", finalBill);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());

        for (int i = 0; i < N; i++) {
            int customerId = Integer.parseInt(sc.nextLine().trim());
            String customerName = sc.nextLine().trim();
            double unitsConsumed = Double.parseDouble(sc.nextLine().trim());

            Customer customer = new Customer(customerId, customerName,
            unitsConsumed);
            customer.display();
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10



# Rajalakshmi Engineering College

Name: Nithilan M  
Email: 241001155@rajalakshmi.edu.in  
Roll no: 241001155  
Phone: 8122020972  
Branch: REC  
Department: IT - Section 4  
Batch: 2028  
Degree: B.E - IT

Scan to verify results



## 2024\_28\_III\_OOPS Using Java Lab

### 2028\_REC\_OOPS using Java\_Week 5\_PAH

Attempt : 1  
Total Mark : 50  
Marks Obtained : 50

#### Section 1 : Coding

##### 1. Problem Statement

Each customer at the bank has an Account Number, Customer Name, and an Initial Balance. The bank allows two types of transactions:

Deposit – Increases the balance. Withdrawal – Decreases the balance, but only if enough funds are available. If the withdrawal amount exceeds the available balance, the transaction should be skipped, and the balance should remain unchanged.

You are required to implement this banking system by:

Creating a class with the necessary attributes to store account details.

Using a constructor to initialize the account details when a new account is created. Providing setter methods to update the details if required. Providing getter methods to retrieve account details. Creating

objects of this class to represent different customers, where each customer can perform deposits and withdrawals.

Instructions:

Implement the class to store account details. Implement the logic for performing deposit and withdrawal transactions. Ensure that withdrawals don't exceed the available balance. After performing the transactions, print the account number, customer name, and final balance.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).
- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

### ***Output Format***

For each customer, print the details in the following format:

1. Account Number: <account\_number>
2. Customer Name: <customer\_name>
3. Final Balance: <final\_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1234

Rahul Sharma

5000

2000

3000

Output: Account Number: 1234

Customer Name: Rahul Sharma  
Final Balance: 4000.0

**Answer**

```
import java.util.Scanner;
```

```
class Account {
```

```
    private int accountNumber;  
    private String customerName;  
    private double balance;
```

```
    public Account(int accountNumber, String customerName, double balance) {  
        this.accountNumber = accountNumber;  
        this.customerName = customerName;  
        this.balance = balance;  
    }
```

```
    public void setAccountNumber(int accountNumber) {  
        this.accountNumber = accountNumber;  
    }
```

```
    public void setCustomerName(String customerName) {  
        this.customerName = customerName;  
    }
```

```
    public void setBalance(double balance) {  
        this.balance = balance;  
    }
```

```
    public int getAccountNumber() {  
        return accountNumber;  
    }
```

```
    public String getCustomerName() {  
        return customerName;  
    }
```

```
    public double getBalance() {  
        return balance;  
    }
```

```
    public void deposit(double amount) {
```

```

        if (amount >= 0) balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= balance) balance -= amount;
    }
}

class CityBankApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());
        for (int i = 0; i < n; i++) {
            int accNo = Integer.parseInt(sc.nextLine());
            String name = sc.nextLine();
            double initBal = Double.parseDouble(sc.nextLine());
            double deposit = Double.parseDouble(sc.nextLine());
            double withdraw = Double.parseDouble(sc.nextLine());

            Account acc = new Account(accNo, name, initBal);
            acc.deposit(deposit);
            acc.withdraw(withdraw);

            System.out.println("Account Number: " + acc.getAccountNumber());
            System.out.println("Customer Name: " + acc.getCustomerName());
            System.out.println("Final Balance: " + acc.getBalance());
        }
        sc.close();
    }
}

```

**Status :** Correct

**Marks : 10/10**

## 2. Problem Statement

Ravi is working as a developer for SecureLogin Systems, which wants to build a system to evaluate the strength of user passwords.

Each user record has:

User ID (integer) User Name (string) Password (string)

The system must calculate whether a password is strong or weak.

A password is considered strong if it meets all of the following conditions:

At least 8 characters long. Contains at least one uppercase letter. Contains at least one lowercase letter. Contains at least one digit. Contains at least one special character (from !@#\$%^&\*).

Ravi has been asked to implement this system using:

A class with attributes for user details. A constructor to initialize user details. Getter and setter methods to retrieve or update user details. A method to check whether the password is strong. Objects of the class to represent users.

Finally, display each user's details and indicate whether their password is Strong or Weak.

### ***Input Format***

The first line contains an integer N, representing the number of users.

For each user:

The next line contains the User ID (integer).

The next line contains the User Name (string).

The next line contains the Password (string).

### ***Output Format***

For each user, print the details in the following format:

User ID: <user\_id>

User Name: <user\_name>

Password: <password>

Password Strength: <Strong/Weak>

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1001

Ravi Kumar

Abc@1234

Output: User ID: 1001

User Name: Ravi Kumar

Password: Abc@1234

Password Strength: Strong

### **Answer**

```
import java.util.*;
```

```
class User {  
    private int userId;  
    private String userName;  
    private String password;  
  
    public User(int userId, String userName, String password) {  
        this.userId = userId;  
        this.userName = userName;  
        this.password = password;  
    }  
  
    public int getUserId() {  
        return userId;  
    }  
  
    public String getUserName() {  
        return userName;  
    }  
  
    public String getPassword() {  
        return password;  
    }  
  
    public void setUserId(int userId) {
```

```

        this.userId = userId;
    }

    public void setUsername(String userName) {
        this.userName = userName;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String checkPasswordStrength() {
        if (password.length() < 8) return "Weak";

        boolean hasUpper = false, hasLower = false, hasDigit = false, hasSpecial = false;
        String specialChars = "!@#$%^&*";

        for (char ch : password.toCharArray()) {
            if (Character.isUpperCase(ch)) hasUpper = true;
            else if (Character.isLowerCase(ch)) hasLower = true;
            else if (Character.isDigit(ch)) hasDigit = true;
            else if (specialChars.indexOf(ch) != -1) hasSpecial = true;
        }

        if (hasUpper && hasLower && hasDigit && hasSpecial) return "Strong";
        else return "Weak";
    }

    public void display() {
        System.out.println("User ID: " + userId);
        System.out.println("User Name: " + userName);
        System.out.println("Password: " + password);
        System.out.println("Password Strength: " + checkPasswordStrength());
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());
    }
}

```

```

    for (int i = 0; i < N; i++) {
        int userId = Integer.parseInt(sc.nextLine().trim());
        String userName = sc.nextLine().trim();
        String password = sc.nextLine().trim();

        User user = new User(userId, userName, password);
        user.display();
    }

    sc.close();
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Neha is working as a developer for CityQuiz Platform, which wants to build a system to calculate quiz scores and identify top scorers among participants.

Each participant's record has:

Participant ID (integer) Participant Name (string) An array of scores in 5 quiz rounds (integers, each between 0 and 100)

The system must calculate:

Total Score = sum of scores in all 5 rounds. Average Score = Total Score ÷ 5. If a participant scores above 80 in all rounds, a bonus of 10 points is added to the total score. Identify the Top Scorer among all participants. If two participants have the same total score, the one with the lower Participant ID is considered the top scorer.

Neha has been asked to implement this system using:

A class with attributes for participant details. A constructor to initialize participant details. Getter and setter methods to retrieve or update participant details. A method to calculate total score and average score (including bonus if applicable). Objects of the class to represent



participants.

Finally, display each participant's details and announce the Top Scorer.

### ***Input Format***

The first line of input contains an integer N, representing the number of participants.

For each participant:

- Next line: Participant ID (integer)
- Next line: Participant Name (string)
- Next line: 5 integers separated by spaces (scores for 5 quiz rounds)

### ***Output Format***

For each participant:

- Participant ID: <participant\_id>
- Participant Name: <participant\_name>
- Total Score: <total\_score>
- Average Score: <average\_score>

Finally, print "Top Scorer: <participant\_name> with <total\_score> points"

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

85 90 88 92 87

Output: Participant ID: 1001

Participant Name: Ravi Kumar

Total Score: 452

Average Score: 90

Top Scorer: Ravi Kumar with 452 points

**Answer**

```
import java.util.*;

class Participant {
    private int participantId;
    private String participantName;
    private int[] scores;
    private int totalScore;
    private int averageScore;

    public Participant(int participantId, String participantName, int[] scores) {
        this.participantId = participantId;
        this.participantName = participantName;
        this.scores = scores;
        calculateScores();
    }

    public int getParticipantId() {
        return participantId;
    }

    public String getParticipantName() {
        return participantName;
    }

    public int getTotalScore() {
        return totalScore;
    }

    public int getAverageScore() {
        return averageScore;
    }

    public void setScores(int[] scores) {
        this.scores = scores;
        calculateScores();
    }

    private void calculateScores() {
        int sum = 0;
```

```

        boolean bonusEligible = true;
        for (int score : scores) {
            sum += score;
            if (score <= 80) {
                bonusEligible = false;
            }
        }
        if (bonusEligible) {
            sum += 10;
        }
        totalScore = sum;
        averageScore = sum / scores.length;
    }

    public void display() {
        System.out.println("Participant ID: " + participantId);
        System.out.println("Participant Name: " + participantName);
        System.out.println("Total Score: " + totalScore);
        System.out.println("Average Score: " + averageScore);
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());

        Participant topScorer = null;
        List<Participant> participants = new ArrayList<>();

        for (int i = 0; i < N; i++) {
            int id = Integer.parseInt(sc.nextLine().trim());
            String name = sc.nextLine().trim();
            String[] scoreStr = sc.nextLine().trim().split("\\s+");
            int[] scores = new int[5];
            for (int j = 0; j < 5; j++) {
                scores[j] = Integer.parseInt(scoreStr[j]);
            }

            Participant p = new Participant(id, name, scores);
            participants.add(p);
        }
    }
}

```

```

        if (topScorer == null ||
            p.getTotalScore() > topScorer.getTotalScore() ||
            (p.getTotalScore() == topScorer.getTotalScore() && p.getParticipantId()
            < topScorer.getParticipantId())) {
            topScorer = p;
        }
    }

    for (Participant p : participants) {
        p.display();
    }

    System.out.println("Top Scorer: " + topScorer.getParticipantName() + " with "
    + topScorer.getTotalScore() + "points");
    sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Anjali is working as a developer for CityFitness Gym, which wants to build a system to calculate monthly membership fees for gym members based on the type of membership and the number of personal training sessions booked.

Each member's record has:

Member ID (integer) Member Name (string) Membership Type (string: "Basic", "Premium", "Elite") Number of Personal Training Sessions (integer)

The monthly fees are:

Basic – 1000 units Premium – 1500 units Elite – 2000 units

The cost of personal training sessions is 500 units per session.

The calculation rules:

Total Amount = Membership Fee + (Number of Personal Training Sessions × 500) If the number of sessions is more than 5, a 10% discount is applied on the total amount. If the member has Elite membership and the total amount exceeds 4000, an additional 5% service tax is added after discount.

Anjali has been asked to implement this system using:

A class with attributes for member details. A constructor to initialize member details. Getter and Setter methods to retrieve and update member details if required. A method to calculate the final monthly fee. Objects of the class to represent members.

Finally, display each member's details and the final monthly fee.

### ***Input Format***

The first line contains an integer N, representing the number of members.

For each member:

- Next line contains Member ID (integer)
- Next line contains Member Name (string)
- Next line contains Membership Type ("Basic", "Premium", "Elite")
- Next line contains Number of Personal Training Sessions (integer)

### ***Output Format***

For each member, print:

- Member ID: <member\_id>
- Member Name: <member\_name>
- Final Monthly Fee: <final\_fee> (The final fee must be rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

Basic

3

Output: Member ID: 1001

Member Name: Ravi Kumar

Final Monthly Fee: 2500.0

**Answer**

```
import java.util.*;
```

```
class Member {  
    private int memberId;  
    private String memberName;  
    private String membershipType;  
    private int personalSessions;  
    private double finalFee;  
  
    public Member(int memberId, String memberName, String membershipType,  
int personalSessions) {  
        this.memberId = memberId;  
        this.memberName = memberName;  
        this.membershipType = membershipType;  
        this.personalSessions = personalSessions;  
        this.finalFee = calculateFinalFee();  
    }  
  
    public int getMemberId() {  
        return memberId;  
    }  
  
    public String getMemberName() {  
        return memberName;  
    }  
  
    public String getMembershipType() {  
        return membershipType;  
    }  
  
    public int getPersonalSessions() {  
        return personalSessions;  
    }  
  
    public double getFinalFee() {
```

```
        return finalFee;
    }

    public void setMemberId(int memberId) {
        this.memberId = memberId;
    }

    public void setMemberName(String memberName) {
        this.memberName = memberName;
    }

    public void setMembershipType(String membershipType) {
        this.membershipType = membershipType;
        this.finalFee = calculateFinalFee();
    }

    public void setPersonalSessions(int personalSessions) {
        this.personalSessions = personalSessions;
        this.finalFee = calculateFinalFee();
    }

    private double calculateFinalFee() {
        double membershipFee = 0;
        switch (membershipType) {
            case "Basic":
                membershipFee = 1000;
                break;
            case "Premium":
                membershipFee = 1500;
                break;
            case "Elite":
                membershipFee = 2000;
                break;
        }

        double total = membershipFee + (personalSessions * 500);

        if (personalSessions > 5) {
            total = total - (total * 0.10);
        }

        if (membershipType.equals("Elite") && total > 4000) {
```

```

        total = total + (total * 0.05);
    }

    return total;
}

public void display() {
    System.out.println("Member ID: " + memberId);
    System.out.println("Member Name: " + memberName);
    System.out.printf("Final Monthly Fee: %.1f%n", finalFee);
}
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());

        for (int i = 0; i < N; i++) {
            int memberId = Integer.parseInt(sc.nextLine().trim());
            String memberName = sc.nextLine().trim();
            String membershipType = sc.nextLine().trim();
            int personalSessions = Integer.parseInt(sc.nextLine().trim());

            Member member = new Member(memberId, memberName,
            membershipType, personalSessions);
            member.display();
        }

        sc.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Neha is working as a developer for CityMovie Theatre, which wants to build a system to calculate total ticket cost for movie-goers based on the number of tickets and type of seats booked.



Each customer's booking has:

Booking ID (integer) Customer Name (string) Number of Tickets (integer) Seat Type (string: "Standard", "Premium", "VIP")

The ticket prices are:

Standard – 250 units per ticket Premium – 400 units per ticket VIP – 600 units per ticket

The calculation rules:

Total Amount = Number of Tickets × Seat Price

If a customer books more than 4 tickets, they get a 10% discount on the total amount.

If the booking is for VIP seats and the total amount exceeds 3000 units, a 5% luxury tax is added after any discount.

Neha has been asked to implement this system using:

A class with attributes for booking details. A constructor to initialize booking details. Getter and Setter methods to retrieve and update booking details if required. A method to calculate the final ticket cost. Objects of the class to represent bookings.

Finally, display each customer's details and final ticket amount.

### ***Input Format***

The first line contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the Booking ID (integer).
- The next line contains the Customer Name (string).
- The next line contains Number of Tickets (integer).
- The next line contains Seat Type ("Standard", "Premium", or "VIP").

### ***Output Format***

For each booking, print:

- Booking ID: <booking\_id>
- Customer Name: <customer\_name>
- Final Ticket Amount: <final\_amount> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1001

Ravi Kumar

3

Standard

Output: Booking ID: 1001

Customer Name: Ravi Kumar

Final Ticket Amount: 750.0

### **Answer**

```
import java.util.*;
```

```
class Booking {
```

```
    private int bookingId;
```

```
    private String customerName;
```

```
    private int numTickets;
```

```
    private String seatType;
```

```
    private double finalAmount;
```

```
    public Booking(int bookingId, String customerName, int numTickets, String  
seatType) {
```

```
        this.bookingId = bookingId;
```

```
        this.customerName = customerName;
```

```
        this.numTickets = numTickets;
```

```
        this.seatType = seatType;
```

```
        this.finalAmount = calculateFinalAmount();
```

```
    }
```

```
    public int getBookingId() {
```

```
        return bookingId;
```

```
    }
```

```
public String getCustomerName() {  
    return customerName;  
}
```

```
public int getNumTickets() {  
    return numTickets;  
}
```

```
public String getSeatType() {  
    return seatType;  
}
```

```
public double getFinalAmount() {  
    return finalAmount;  
}
```

```
public void setBookingId(int bookingId) {  
    this.bookingId = bookingId;  
}
```

```
public void setCustomerName(String customerName) {  
    this.customerName = customerName;  
}
```

```
public void setNumTickets(int numTickets) {  
    this.numTickets = numTickets;  
    this.finalAmount = calculateFinalAmount();  
}
```

```
public void setSeatType(String seatType) {  
    this.seatType = seatType;  
    this.finalAmount = calculateFinalAmount();  
}
```

```
private double calculateFinalAmount() {  
    double pricePerTicket = 0;  
    switch (seatType) {  
        case "Standard":  
            pricePerTicket = 250;  
            break;  
        case "Premium":
```

```

        pricePerTicket = 400;
        break;
    case "VIP":
        pricePerTicket = 600;
        break;
    }

    double total = numTickets * pricePerTicket;

    if (numTickets > 4) {
        total = total - (total * 0.10);
    }

    if (seatType.equals("VIP") && total > 3000) {
        total = total + (total * 0.05);
    }

    return total;
}

public void display() {
    System.out.println("Booking ID: " + bookingId);
    System.out.println("Customer Name: " + customerName);
    System.out.printf("Final Ticket Amount: %.1f%n", finalAmount);
}
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());

        for (int i = 0; i < N; i++) {
            int bookingId = Integer.parseInt(sc.nextLine().trim());
            String customerName = sc.nextLine().trim();
            int numTickets = Integer.parseInt(sc.nextLine().trim());
            String seatType = sc.nextLine().trim();

            Booking booking = new Booking(bookingId, customerName, numTickets,
            seatType);
            booking.display();
        }
    }
}

```

```
241001155
    sc.close();
  }
}
```

**Status :** Correct

**Marks : 10/10**

# Rajalakshmi Engineering College

Name: Nithilan M  
Email: 241001155@rajalakshmi.edu.in  
Roll no: 241001155  
Phone: 8122020972  
Branch: REC  
Department: IT - Section 4  
Batch: 2028  
Degree: B.E - IT

Scan to verify results



## 2024\_28\_III\_OOPS Using Java Lab

### 2028\_REC\_OOPS using Java\_Week 5\_Q5

Attempt : 1  
Total Mark : 10  
Marks Obtained : 10

#### Section 1 : Coding

##### 1. Problem Statement

Ram is working as a developer for BrightEdu Coaching Center, which wants to build a student fee management system.

Each student's enrollment has:

An Enrollment ID (integer) A Student Name (string) The Number of Subjects (integer)

The fee calculation rules are:

Registration Fee = 1000 units (flat for every student). Per Subject Fee = 800 units. If the student enrolls in more than 5 subjects, a 20% scholarship (discount) is applied on the total fee.

Ram has been asked to implement this system using:

A class with attributes for student details. A constructor to initialize student details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent student enrollments.

Finally, display each student's details and final fee.

### ***Input Format***

The first line of input contains an integer N, representing the number of students.

For each student:

- The next line contains the Enrollment ID (integer).
- The following line contains the student's name (string).
- The next line contains the Number of subjects (integer).

### ***Output Format***

For each student, print the details in the following format:

- Enrollment ID: <enrollment\_id>
- Student Name: <student\_name>
- Final Fee: <final\_fee> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1234

Ravi Kumar

3

Output: Enrollment ID: 1234

Student Name: Ravi Kumar

Final Fee: 3400.0

### ***Answer***

```
import java.util.*;
```

```
class Student {  
    private int enrollmentId;
```

```
private String studentName;
private int numSubjects;
private double finalFee;

public Student(int enrollmentId, String studentName, int numSubjects) {
    this.enrollmentId = enrollmentId;
    this.studentName = studentName;
    this.numSubjects = numSubjects;
    this.finalFee = calculateFee(numSubjects);
}
```

```
public int getEnrollmentId() {
    return enrollmentId;
}
```

```
public String getStudentName() {
    return studentName;
}
```

```
public int getNumSubjects() {
    return numSubjects;
}
```

```
public double getFinalFee() {
    return finalFee;
}
```

```
public void setEnrollmentId(int enrollmentId) {
    this.enrollmentId = enrollmentId;
}
```

```
public void setStudentName(String studentName) {
    this.studentName = studentName;
}
```

```
public void setNumSubjects(int numSubjects) {
    this.numSubjects = numSubjects;
    this.finalFee = calculateFee(numSubjects);
}
```

```
private double calculateFee(int numSubjects) {
    double fee = 1000 + (numSubjects * 800);
}
```



```

        if (numSubjects > 5) {
            fee = fee - (fee * 0.20);
        }
        return fee;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = Integer.parseInt(sc.nextLine().trim());

        for (int i = 0; i < N; i++) {
            int enrollmentId = Integer.parseInt(sc.nextLine().trim());
            String studentName = sc.nextLine().trim();
            int numSubjects = Integer.parseInt(sc.nextLine().trim());

            Student student = new Student(enrollmentId, studentName,
numSubjects);

            System.out.println("Enrollment ID: " + student.getEnrollmentId());
            System.out.println("Student Name: " + student.getStudentName());
            System.out.printf("Final Fee: %.1f%n", student.getFinalFee());
        }

        sc.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

# Rajalakshmi Engineering College

Name: Nithilan M  
Email: 241001155@rajalakshmi.edu.in  
Roll no: 241001155  
Phone: 8122020972  
Branch: REC  
Department: IT - Section 4  
Batch: 2028  
Degree: B.E - IT

Scan to verify results



## 2024\_28\_III\_OOPS Using Java Lab

### 2028\_REC\_OOPS using Java\_Week 5\_Q4

Attempt : 1  
Total Mark : 10  
Marks Obtained : 10

#### Section 1 : Coding

##### 1. Problem Statement

You are working as a developer for CityCab, a taxi service company that wants to build a ride fare management system.

Each customer booking has:

A Booking ID (integer) A Customer Name (string) A Distance Travelled in km (double)

The fare calculation rules are:

Base Fare = 50 units (flat charge for every ride). Per km charge = 10 units/km. If the distance is greater than 20 km, a 10% discount is applied on the total fare.

You are required to implement this system using:

A class with attributes for booking details. A constructor to initialize booking details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customer rides.

Finally, display each booking's details and final fare.

### ***Input Format***

The first line of input contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the booking ID (integer).
- The following line contains the customer's name (string).
- The next line contains the distance travelled (double).

### ***Output Format***

For each booking, print the details in the following format:

1. Booking ID: <booking\_id>
2. Customer Name: <customer\_name>
3. Final Fare: <final\_fare> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1234

Rahul Sharma

15

Output: Booking ID: 1234

Customer Name: Rahul Sharma

Final Fare: 200.0

### ***Answer***

```
import java.util.Scanner;
```

```
class Booking {
```

```
private int bookingId;  
private String customerName;  
private double distance;  
private double fare;
```

```
public Booking(int bookingId, String customerName, double distance) {  
    this.bookingId = bookingId;  
    this.customerName = customerName;  
    this.distance = distance;  
    calculateFare();  
}
```

```
public void setBookingId(int bookingId) {  
    this.bookingId = bookingId;  
}
```

```
public void setCustomerName(String customerName) {  
    this.customerName = customerName;  
}
```

```
public void setDistance(double distance) {  
    this.distance = distance;  
    calculateFare();  
}
```

```
public int getBookingId() {  
    return bookingId;  
}
```

```
public String getCustomerName() {  
    return customerName;  
}
```

```
public double getDistance() {  
    return distance;  
}
```

```
public double getFare() {  
    return fare;  
}
```

```
private void calculateFare() {
```

```
        fare = 50 + distance * 10;
        if (distance > 20) {
            fare = fare - (fare * 0.1);
        }
    }
}

class CityCabApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());
        for (int i = 0; i < n; i++) {
            int id = Integer.parseInt(sc.nextLine());
            String name = sc.nextLine();
            double distance = Double.parseDouble(sc.nextLine());
            Booking booking = new Booking(id, name, distance);
            System.out.println("Booking ID: " + booking.getBookingId());
            System.out.println("Customer Name: " + booking.getCustomerName());
            System.out.printf("Final Fare: %.1f\n", booking.getFare());
        }
        sc.close();
    }
}
```

**Status :** Correct

**Marks : 10/10**

# Rajalakshmi Engineering College

Name: Nithilan M  
Email: 241001155@rajalakshmi.edu.in  
Roll no: 241001155  
Phone: 8122020972  
Branch: REC  
Department: IT - Section 4  
Batch: 2028  
Degree: B.E - IT

Scan to verify results



## 2024\_28\_III\_OOPS Using Java Lab

### 2028\_REC\_OOPS using Java\_Week 5\_Q3

Attempt : 1  
Total Mark : 10  
Marks Obtained : 10

#### Section 1 : Coding

##### 1. Problem Statement

Neha is working as a developer for CityElectricity Board, which wants to build a household electricity billing system.

Each customer's electricity account has:

A Customer ID (integer) A Customer Name (string) Units Consumed (double)

The electricity bill is calculated based on these rules:

For the first 100 units 5 units charge per unit For the next 100 units (101–200) 7 units charge per unit For units above 200 10 units charge per unit If the total bill exceeds 2000 units, a 5% discount is applied on the final bill.

Neha has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

### ***Output Format***

For each customer, print the details in the following format:

Customer ID: <customer\_id>

Customer Name: <customer\_name>

Final Bill: <final\_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

80

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 400.0

### ***Answer***

```
import java.util.Scanner;
```

```
class eleacc{
    int id;
    String name;
    double units;

    eleacc(int id,String name,double units){
        this.id=id;
        this.name=name;
        this.units=units;
    }

    double bill(double units){
        double bill=0;
        if(units<=100){
            bill=units*5.0;
        }
        else if(units<=200){
            bill=100*5.0+(units-100)*7.0;
        }
        else if(units>200){
            bill=100.0*5.0+(100.0*7.0)+((units-200.0)*10.0);
        }
        if(bill>2000){
            bill=bill-bill*(0.05);
        }
        return bill;
    }

    void display(int id,String name,double bill){
        System.out.println("Customer ID: "+id);
        System.out.println("Customer Name: "+name);
        System.out.println("Final Bill: "+bill);
    }
}
```

```
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
```



```
int n=sc.nextInt(),id;  
String name;  
double units;  
for(int i=0;i<n;i++){  
    id=sc.nextInt();  
    sc.nextLine();  
    name=sc.nextLine();  
    units=sc.nextDouble();  
    eleacc cust=new eleacc(id,name,units);  
    cust.display(id,name,cust.bill(units));  
}  
}
```

**Status :** Correct

**Marks : 10/10**

# Rajalakshmi Engineering College

Name: Nithilan M  
Email: 241001155@rajalakshmi.edu.in  
Roll no: 241001155  
Phone: 8122020972  
Branch: REC  
Department: IT - Section 4  
Batch: 2028  
Degree: B.E - IT

Scan to verify results



## 2024\_28\_III\_OOPS Using Java Lab

### 2028\_REC\_OOPS using Java\_Week 5\_Q2

Attempt : 1  
Total Mark : 10  
Marks Obtained : 10

#### Section 1 : Coding

##### 1. Problem Statement

You are working as a developer for CityBank, which wants to build a basic account management system.

Each customer at the bank has:

An Account Number (integer) A Customer Name (string) An Initial Balance (double)

The bank allows two types of transactions:

Deposit – increases the balance. Withdrawal – decreases the balance only if enough funds are available.

If the withdrawal amount is greater than the balance, the withdrawal should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for account details. A constructor to initialize account details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's account details after all transactions.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).
- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

### ***Output Format***

For each customer, print the details in the following format:

1. Account Number: <account\_number>
2. Customer Name: <customer\_name>
3. Final Balance: <final\_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1234

Rahul Sharma

5000

2000

3000

Output: Account Number: 1234

Customer Name: Rahul Sharma

Final Balance: 4000.0

**Answer**

```
import java.util.*;
```

```
class Account {  
    private int accountNumber;  
    private String customerName;  
    private double balance;  
  
    public Account(int accountNumber, String customerName, double balance) {  
        this.accountNumber = accountNumber;  
        this.customerName = customerName;  
        this.balance = balance;  
    }  
  
    public int getAccountNumber() {  
        return accountNumber;  
    }  
  
    public String getCustomerName() {  
        return customerName;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void setAccountNumber(int accountNumber) {  
        this.accountNumber = accountNumber;  
    }  
  
    public void setCustomerName(String customerName) {  
        this.customerName = customerName;  
    }  
  
    public void setBalance(double balance) {  
        this.balance = balance;  
    }  
  
    public void deposit(double amount) {  
        if (amount >= 0) {
```

```

        balance += amount;
    }
}

public void withdraw(double amount) {
    if (amount <= balance) {
        balance -= amount;
    }
}
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine().trim());
        for (int i = 0; i < N; i++) {
            int accountNumber = Integer.parseInt(sc.nextLine().trim());
            String customerName = sc.nextLine().trim();
            double initialBalance = Double.parseDouble(sc.nextLine().trim());
            double depositAmount = Double.parseDouble(sc.nextLine().trim());
            double withdrawAmount = Double.parseDouble(sc.nextLine().trim());
            Account acc = new Account(accountNumber, customerName,
initialBalance);
            acc.deposit(depositAmount);
            acc.withdraw(withdrawAmount);
            System.out.println("Account Number: " + acc.getAccountNumber());
            System.out.println("Customer Name: " + acc.getCustomerName());
            System.out.printf("Final Balance: %.1f%n", acc.getBalance());
        }
        sc.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

# Rajalakshmi Engineering College

Name: Nithilan M  
Email: 241001155@rajalakshmi.edu.in  
Roll no: 241001155  
Phone: 8122020972  
Branch: REC  
Department: IT - Section 4  
Batch: 2028  
Degree: B.E - IT

Scan to verify results



## 2024\_28\_III\_OOPS Using Java Lab

### 2028\_REC\_OOPS using Java\_Week 5\_MCQ

Attempt : 1  
Total Mark : 15  
Marks Obtained : 15

#### Section 1 : MCQ

1. What is the output of the following code?

```
class Box {  
    int height;  
    Box(int height) {  
        this.height = height;  
    }  
    void modifyHeight(Box b) {  
        b.height += 10;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Box b1 = new Box(20);  
        b1.modifyHeight(b1);  
        System.out.println(b1.height);  
    }  
}
```

**Answer**

30

**Status :** Correct

**Marks :** 1/1

2. What will be the output of the following code?

```
class A {  
    int y = 30;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        A a1 = new A();  
        A a2 = new A();  
        a1.y = 50;  
        System.out.println(a2.y);  
    }  
}
```

**Answer**

30

**Status :** Correct

**Marks :** 1/1

3. What will be the output of the following code?

```
class Person {  
    int age = 18;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.age += 2;  
        System.out.println("Age: " + p.age);  
    }  
}
```

**Answer**

Age: 20

**Status :** Correct

**Marks :** 1/1

4. What will be the output of the following code?

```
class Ball {  
    int size = 11;  
}  
  
class Game {  
    public static void main(String[] args) {  
        Ball b1 = new Ball();  
        Ball b2 = new Ball();  
        b2.size = 10;  
        System.out.println(b1.size);  
    }  
}
```

**Answer**

11

**Status :** Correct

**Marks :** 1/1

5. What will be the output of the following code?

```
class Test {  
    private int value;  
    Test(int value) {  
        this.value = value;  
    }  
    public int getValue() {  
        return value;  
    }  
}
```



```
public class Main {  
    public static void main(String[] args) {  
        Test obj = new Test(10);  
        System.out.println(obj.value);  
    }  
}
```

**Answer**

Compile-time error

**Status :** Correct

**Marks :** 1/1

6. What will be the output of the following code?

```
class Person {  
    String name;  
    void setName(String n) {  
        name = n;  
    }  
    void printName() {  
        System.out.println(name);  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.printName();  
    }  
}
```

**Answer**

null

**Status :** Correct

**Marks :** 1/1

7. What will be the output of the following code?

```
class MathUtils {
```

```
int add(int x) {  
    return x + x;  
}  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MathUtils m = new MathUtils();  
        System.out.println(m.add(5));  
    }  
}
```

**Answer**

10

**Status :** Correct

**Marks :** 1/1

8. What will be the output of the following code?

```
class Sample {  
    int x = 10;  
  
    void display() {  
        System.out.println("x = " + x);  
    }  
  
    public static void main(String[] args) {  
        Sample s = new Sample();  
        s.display();  
    }  
}
```

**Answer**

x = 10

**Status :** Correct

**Marks :** 1/1

9. What will be the output of the following code?

```
class Box {  
    int length = 5;  
    int width = 4;  
  
    int area() {  
        return length * width;  
    }  
  
    public static void main(String[] args) {  
        Box b = new Box();  
        System.out.println("Area = " + b.area());  
    }  
}
```

**Answer**

Area = 20

**Status :** Correct

**Marks :** 1/1

10. What will be the output of the following code?

```
class Box {  
    int volume(int l, int b, int h) {  
        return l * b * h;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Box b = new Box();  
        System.out.println(b.volume(2, 3, 4));  
    }  
}
```

**Answer**

24

**Status :** Correct

**Marks :** 1/1

11. What will be the output of the following code?

```
class Demo {  
    void printMessage() {  
        System.out.println("Hello from Demo");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.printMessage();  
    }  
}
```

**Answer**

Hello from Demo

**Status :** Correct

**Marks :** 1/1

12. What will be the output of the following code?

```
class A {  
    int val = 20;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        A obj1 = new A();  
        A obj2 = obj1;  
        obj2.val += 5;  
        System.out.println(obj1.val);  
    }  
}
```

**Answer**

25

**Status :** Correct

**Marks :** 1/1

13. What will be the output of the following code?

```
class A {  
    int x = 50;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        A obj1 = new A();  
        A obj2 = obj1;  
        obj2.x = 100;  
        System.out.println(obj1.x);  
    }  
}
```

**Answer**

100

**Status :** Correct

**Marks :** 1/1

14. What will be the output of the following code?

```
class Alpha {  
    void greet(String name) {  
        System.out.println("Hello" + name);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Alpha obj = new Alpha();  
        obj.greet("Anu");  
    }  
}
```

**Answer**

Hello Anu

**Status :** Correct

**Marks :** 1/1

15. What will be the output of the following code?

```
class A {  
    int p = 5;  
    int q = 2;  
}  
  
class Main {  
    public static void main(String[] args) {  
        A obj = new A();  
        System.out.println(obj.p + obj.q);  
    }  
}
```

**Answer**

7

**Status :** Correct

**Marks :** 1/1